Autonomous Retail Researcher Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	Mohit Meharde	EN22CS301609
2	Nancy Agrawal	EN22CS301634
3	Niharika Panwar	EN22CS301645
4	Nikhil Garg	EN22CS301646
5	Mahima Sahu	EN22CS301570
6	Naman Mangroliya	EN22CS301628

Group Name: Group 10D6

Project Number: AAI-21

Industry Mentor Name: Mr. Suraj Nayak

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2025-2026

1. Problem Statement

In the modern retail ecosystem, e-commerce platforms generate massive volumes of product data, including pricing, ratings, reviews, and category classifications. Retail businesses must continuously monitor competitor activity to remain competitive. However, manual data collection from platforms such as Amazon and Flipkart is inefficient, time-consuming, and non-scalable.

Moreover, price fluctuations occur frequently, and without automated tracking mechanisms, businesses struggle to identify patterns or strategic opportunities. Traditional business intelligence tools often rely on static datasets rather than real-time market data.

Therefore, there exists a need for an automated, intelligent, and scalable retail intelligence system capable of:

- Collecting real-time product data,
- Tracking historical price changes,
- Performing AI-based competitive analysis,
- Generating structured and actionable insights.

The Autonomous Retail Researcher Agent addresses this need through an integrated Agentic AI architecture.

1.2 Project Objectives

The primary objective of the Autonomous Retail Researcher Agent is to design and implement an automated and intelligent retail intelligence system that assists businesses in competitor analysis and strategic decision-making. The system integrates web scraping, cloud-based data management, and AI-driven analytics into a unified platform capable of collecting, analyzing, and presenting retail market insights efficiently.

The major objectives of the project are as follows:

- **Automated Data Collection:** Develop a scraping mechanism to extract structured product information such as name, price, rating, and reviews from e-commerce platforms.
- **Structured Data Storage:** Implement a scalable MongoDB database to store product details and maintain historical price tracking.

- **AI-Based Analysis:** Integrate large language models to generate quick insights and detailed multi-agent analytical reports.
- **Interactive Dashboard:** Provide a user-friendly interface for visualization of product data, price trends, and AI-generated insights.
- **Automated Report Generation:** Enable downloadable PDF reports for business-level decision support.

Additionally, the system is designed with modular architecture, secure API management, and performance optimization mechanisms to support scalability and future enhancements. Collectively, these objectives aim to create a comprehensive retail research platform that reduces manual effort while improving analytical accuracy and efficiency.

2. Scope of the Project

The scope of the Autonomous Retail Researcher Agent includes the following functional boundaries:

2.1 Functional Scope

The functional scope defines the operational boundaries of the system.

The platform includes:

- Automated scraping of product data.
- Storage of structured product records.
- Historical price tracking.
- AI-powered competitive analysis.
- Dashboard-based visualization.
- PDF report generation.

The system supports:

- Product search by keyword.
- Platform selection (Amazon / Flipkart).
- Category filtering.
- Quick and deep analytical modes.

2.2 Technical Scope

The technical scope includes:

- Python-based implementation.
- Selenium-based scraping engine.
- BeautifulSoup-based parsing.
- MongoDB Atlas database.
- REST API-based LLM inference.
- Session-state caching.
- Modular layered architecture.

The system is designed as a monolithic application in its current phase but structured to allow microservices expansion.

2.3 Out-of-Scope Elements

The following features are intentionally excluded in the current version:

- User authentication and role-based access.
- Multi-tenant database separation.
- Direct product purchasing.
- Payment gateway integration.
- Real-time alert notifications.
- Fully distributed cloud deployment.

3. Proposed Solution

The proposed solution is an AI-powered retail intelligence platform designed to automate competitor research and generate actionable insights.

The system follows a layered architecture:

1. Presentation Layer – User interface built using Streamlit.
2. Application Layer – Scraping engine, AI analysis modules, and report generation engine.
3. Data Layer – MongoDB Atlas cloud database.
4. External Services Layer – Integration with Gemini API and Groq LLM API.

The system operates through an end-to-end workflow where user queries trigger automated scraping, data storage, AI analysis, and report generation. The modular design ensures scalability and future extensibility.

4. Key Features

The Autonomous Retail Researcher Agent incorporates a set of advanced and carefully engineered features that collectively enable automated retail intelligence, intelligent decision support, and scalable system design. These features are designed not only to automate data collection but also to ensure reliability, analytical depth, and usability.

4.1 Automated Web Scraping with Retry Logic

One of the foundational features of the system is its automated web scraping engine. The scraping modules are built using Selenium WebDriver for browser automation and BeautifulSoup for HTML parsing. The system dynamically loads product pages and extracts structured data fields in real time.

To ensure robustness and reliability, the scraper integrates a retry mechanism with configurable parameters:

- Maximum retry attempts (default: 3)
- Configurable scraping delay
- Exponential backoff strategy
- Automatic driver management using webdriver-manager

This design reduces the likelihood of data loss due to temporary network failures or dynamic content loading issues. By automating the scraping lifecycle, the system eliminates the need for manual competitor tracking.

4.2 Historical Price Tracking Mechanism

A key analytical capability of the system is historical price tracking. Each time a product is scraped, its current price is appended to a dedicated price_history collection rather than overwriting previous records.

This append-only strategy enables:

- Time-series price trend analysis.
- Identification of seasonal pricing patterns.
- Detection of sudden price drops.
- Long-term competitive strategy evaluation.

By maintaining complete price history, the system supports advanced business intelligence applications.

4.3 Single-Agent Quick Analysis

The system includes a Quick Analysis module powered by a large language model. This module provides rapid insights with minimal latency.

The quick analysis generates:

- Overall price range summary.
- Identification of top-rated products.
- Best value recommendations.
- General competitive positioning insights.

This feature ensures that users receive immediate high-level insights within seconds, making it suitable for quick decision-making scenarios.

4.4 Multi-Agent Deep Reasoning Framework

Beyond simple summarization, the system implements a multi-agent reasoning architecture using CrewAI. This advanced feature enables distributed intelligence across specialized agents.

The deep analysis involves the following agents:

- **Data Scout:** Extracts structured market insights.
- **Pricing Strategist:** Evaluates competitor pricing strategies.
- **Risk Assessor:** Identifies volatility and potential risks.
- **Demand Forecaster:** Analyzes probable demand patterns.
- **Report Writer:** Compiles findings into an executive summary.

The manager agent coordinates task allocation and integrates outputs into a cohesive analytical report. This multi-agent structure enhances reasoning depth and mimics collaborative human expertise.

4.5 Executive PDF Report Generation

To bridge the gap between technical analysis and managerial decision-making, the system generates structured PDF reports using ReportLab.

Each report includes:

- Executive summary
- Price comparison tables
- Strategic recommendations
- Structured section formatting

Reports are stored in the database and can be re-downloaded without regeneration. This ensures persistence and professional documentation.

4.6 Interactive Analytics Dashboard

The Streamlit-based dashboard provides an intuitive interface for user interaction and visualization. It includes:

- KPI summary cards
- Product explorer with filtering
- Price trend line charts
- Platform comparison charts
- AI insight display panels

Visualization libraries such as Plotly and Altair enhance interpretability. This feature ensures that complex analytical outputs are presented in an accessible format.

5. Overall Architecture and Workflow

5.1 System Architecture

The system architecture of the Autonomous Retail Researcher Agent is designed using a layered architectural model to ensure modularity, scalability, maintainability, and separation of concerns. The architecture is divided into four major layers: Presentation Layer, Application Layer, Data Layer, and External Services Layer. Each layer performs a specific function while interacting with other layers through well-defined interfaces.

This structured design ensures that changes in one component do not directly affect other components, thereby improving system stability and future extensibility.

A. Presentation Layer

The Presentation Layer serves as the user-facing interface of the system. It is implemented using the Streamlit framework, which enables rapid development of interactive dashboards and web-based analytics applications.

This layer is responsible for:

- Accepting user search queries (e.g., product keyword and platform selection).
- Displaying scraped product data in tabular format.
- Visualizing price comparisons and historical trends.
- Presenting AI-generated insights.
- Enabling download of PDF reports.

The dashboard includes KPI summary cards such as:

- Average price,
- Minimum and maximum price,
- Top-rated product,
- Platform comparison metrics.

Visualization tools such as Plotly and Altair are integrated to generate dynamic charts, including:

- Line charts for price history,
- Bar charts for price comparison,
- Rating distribution graphs.

The Presentation Layer communicates with the Application Layer through function calls and session state management. It does not directly interact with the database, ensuring a clean

separation between UI logic and business logic.

B. Application Layer

The Application Layer acts as the core processing unit of the system. It contains all business logic, computation modules, and coordination mechanisms.

This layer includes the following components:

1. Scraper Modules

The scraper modules are responsible for extracting real-time product data from e-commerce platforms. Selenium WebDriver automates browser interactions, while BeautifulSoup parses the HTML structure to extract:

- Product name
- Price
- Rating
- Number of reviews
- Product URL

Retry logic and error handling mechanisms are integrated to ensure resilience against dynamic page loading and network interruptions.

2. AI Analysis Engine

The AI analysis engine performs intelligent reasoning on the collected data. It supports two operational modes:

- **Quick Analysis Mode** – Uses a lightweight LLM call for instant market summary.
- **Deep Multi-Agent Mode** – Uses CrewAI framework to coordinate multiple specialized agents for advanced analysis.

The AI engine generates:

- Competitive price positioning insights,
- Risk evaluation summaries,
- Demand estimation commentary,
- Strategic recommendations.

The modular AI structure ensures extensibility for future integration of predictive analytics models.

3. PDF Report Generator

The PDF generation module converts AI-generated markdown reports into professionally formatted PDF documents using the ReportLab library. The reports include:

- Executive summary,
- Structured headings,
- Analytical observations,
- Business recommendations.

Generated reports are stored for persistence and future retrieval.

4. Utility and Configuration Modules

Utility modules handle:

- SHA-256 hashing for product identification,
- Data normalization,
- Session-based caching,
- Environment variable management.

These supporting modules ensure system integrity, performance optimization, and secure configuration handling.

C. Data Layer

The Data Layer is responsible for structured storage and retrieval of all system data. MongoDB Atlas, a cloud-based NoSQL database, is used due to its flexibility and scalability. The database contains three primary collections:

1. Products Collection

Stores static product metadata including:

- Product ID (hashed),
- Platform name,
- Title,
- Category,
- URL.

Upsert operations ensure no duplication occurs.

2. Price History Collection

Maintains time-series data by appending new price entries rather than overwriting previous values. Each entry includes:

- Product ID,
- Price,
- Timestamp.

This structure enables advanced historical trend analysis.

3. Reports Collection

Stores generated AI reports along with metadata such as:

- Query term,
- Generation timestamp,
- Platform,
- Analysis type.

Indexing mechanisms are implemented to optimize retrieval speed and reduce query latency.

D. External Services Layer

The External Services Layer connects the system with third-party APIs and online platforms. This layer integrates:

1. E-commerce Platforms

The scraper interacts with:

- Amazon
- Flipkart

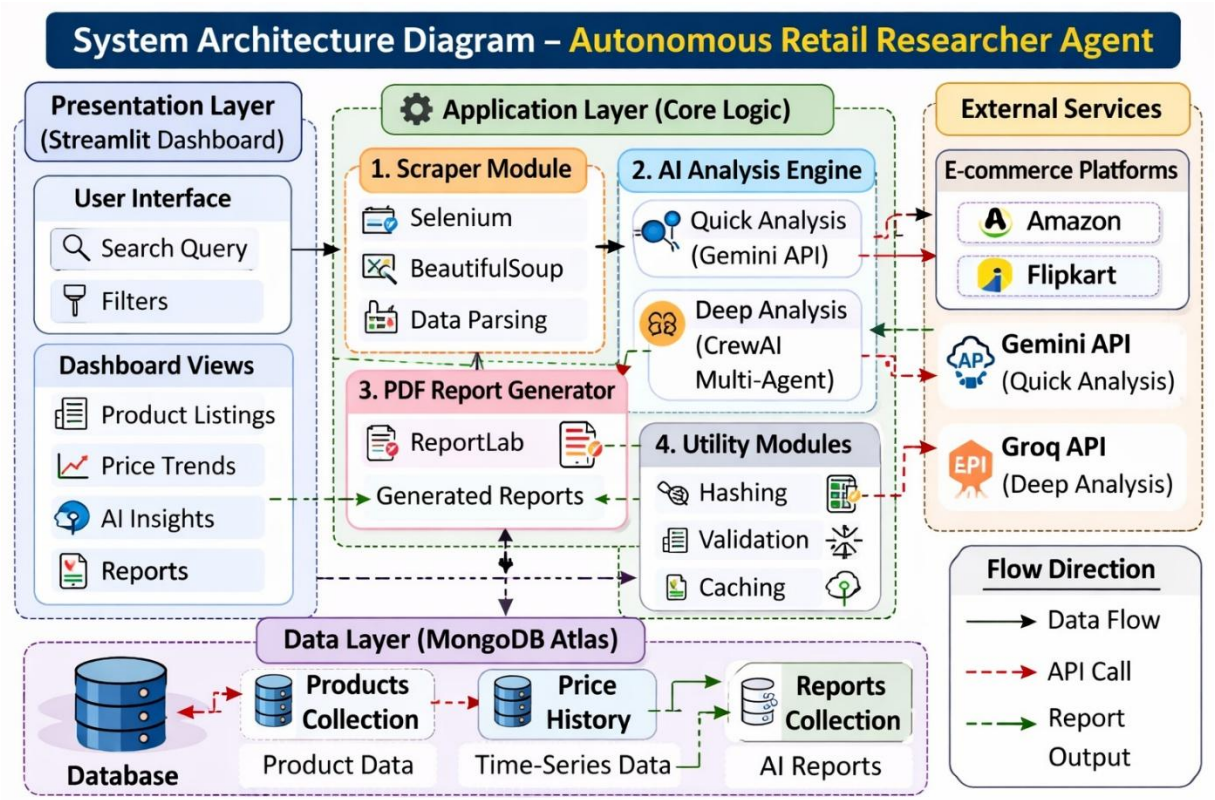
These platforms serve as primary data sources for retail intelligence.

2. Gemini API (Quick AI Analysis)

Used for rapid inference and lightweight market summaries. It provides low-latency responses suitable for real-time dashboard insights.

3. Groq API (Deep AI Analysis)

Used within the multi-agent CrewAI framework for comprehensive analytical reasoning. It supports structured prompt execution and advanced language model processing.



5.2 Workflow Description

The workflow of the Autonomous Retail Researcher Agent is designed as a structured and sequential pipeline that transforms raw web data into actionable business intelligence. The system follows a well-defined multi-stage processing flow, ensuring automation, data integrity, intelligent reasoning, and user-friendly visualization.

Stage 1: User Query Submission

The workflow begins at the Presentation Layer, where the user interacts with the Streamlit dashboard. The user provides:

- Product search keyword (e.g., "wireless headphones")
- Platform selection (Amazon or Flipkart)
- Optional filters such as category or analysis mode

Once the user submits the query, it triggers the backend processing in the Application Layer. The input parameters are validated to ensure correctness before initiating data collection.

Stage 2: Data Collection via Scraper Module

Upon receiving the query, the Scraper Module is activated. Selenium WebDriver opens the target e-commerce platform and navigates dynamically through search result pages. Since most e-commerce websites load content asynchronously, Selenium enables real-time rendering of JavaScript-driven elements.

The system extracts the following attributes:

- Product title
- Current price
- Rating
- Number of reviews
- Product URL

BeautifulSoup is then used to parse the rendered HTML content for structured extraction. Retry logic is applied in case of timeouts or temporary connection issues. This stage ensures accurate and comprehensive data retrieval.

Stage 3: Data Cleaning and Normalization

Raw scraped data often contains inconsistencies such as:

- Currency symbols in price values
- Missing ratings
- Inconsistent text formatting

Therefore, a normalization step is performed to:

- Convert prices into numerical format
- Handle missing or null values
- Standardize product naming conventions
- Remove duplicate entries

Additionally, a SHA-256 hash is generated to create a unique product identifier, ensuring consistent database storage and preventing redundancy.

Stage 4: Data Storage in MongoDB

After validation and normalization, the processed data is stored in MongoDB Atlas. The storage operation includes:

- Upsert into the Products Collection
- Append operation in the Price History Collection
- Timestamp generation for tracking

This structured storage enables long-term price trend analysis and historical comparisons. The use of a NoSQL database provides flexibility in schema management and scalability.

Stage 5: AI Dataset Retrieval

Once data is stored, the AI Analysis Engine retrieves the relevant dataset from MongoDB. The retrieval process may include:

- Filtering by product category
- Sorting by price or rating
- Aggregating summary statistics

The prepared dataset is then formatted into structured prompts for the language model.

Stage 6: Execution of Quick or Deep Analysis

The system supports two modes of AI reasoning:

A. Quick Analysis Mode

In this mode, a single API call is made to generate:

- Overall price range insights
- Best value product identification
- Competitive summary

This mode prioritizes speed and is suitable for real-time decision-making.

B. Deep Multi-Agent Analysis Mode

In deep analysis, multiple AI agents collaborate through a coordinated reasoning framework. Each agent performs a specialized task such as:

- Competitive pricing evaluation
- Risk assessment
- Demand estimation
- Strategic recommendation generation

The outputs are combined into a comprehensive executive-level report.

Stage 7: Structured Insight Generation

The AI engine produces insights in structured markdown format. The report typically includes:

- Executive Summary
- Market Overview
- Competitive Analysis
- Risk and Opportunity Assessment
- Strategic Recommendations

This structured output ensures clarity and readability for business stakeholders.

Stage 8: PDF Report Generation and Storage

The generated markdown content is passed to the PDF generation module. Using ReportLab, the system formats the content into a professional report layout with headings and sections.

The PDF file is:

- Saved locally (temporarily)
- Stored in the Reports Collection in MongoDB
- Made available for download through the dashboard

This step ensures persistence and professional documentation.

Stage 9: Dashboard Visualization and Output Display

Finally, the Presentation Layer updates dynamically to display:

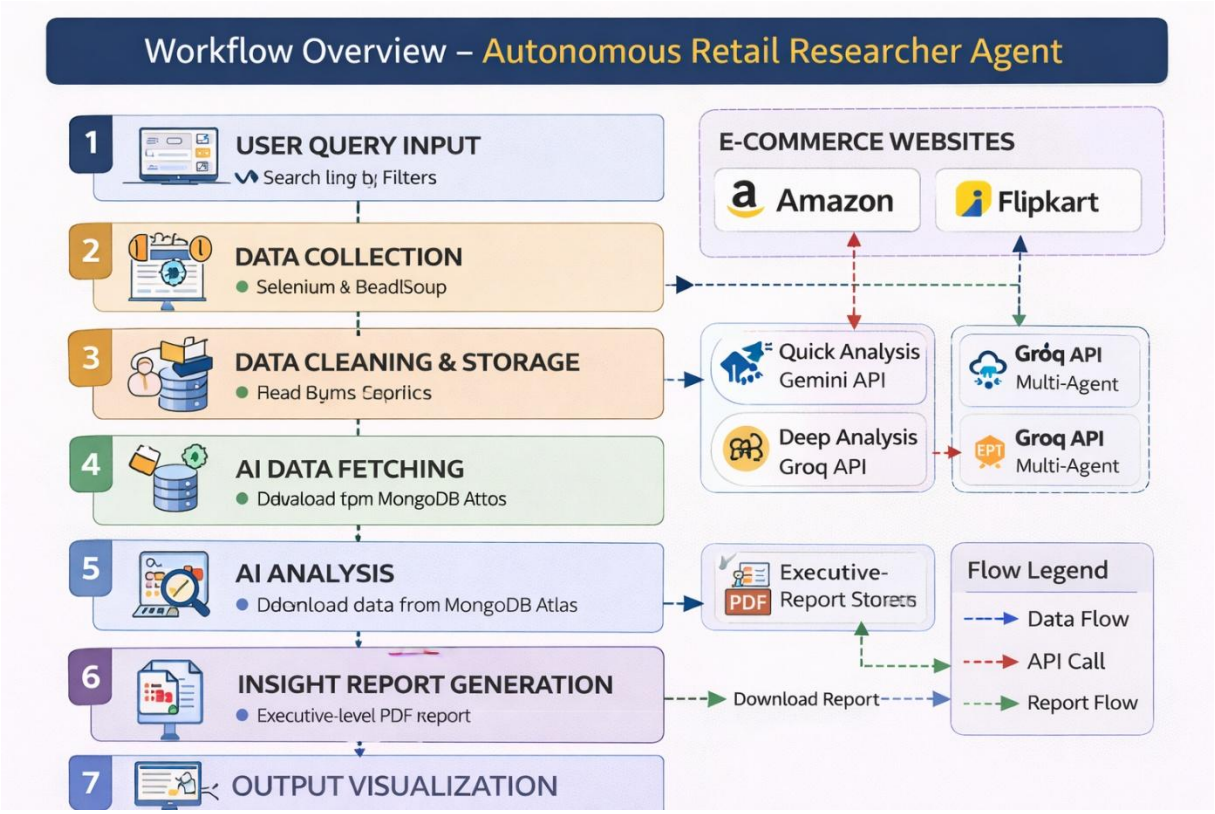
- Product listings
- Price comparison charts
- Historical trend graphs
- AI-generated insights
- Downloadable report links

Caching mechanisms improve performance by preventing redundant API calls during repeated queries.

The overall workflow can be summarized as:

User Input → Data Scraping → Data Cleaning → Database Storage → AI Analysis → Report Generation → Dashboard Visualization

This systematic and layered workflow ensures seamless integration of automation, intelligence, and visualization. It transforms raw retail data into structured business insights efficiently and reliably.



6. Tools & Technologies Used

6.1 Programming Language

6.1.1 Python 3.11+

Python is used as the primary development language due to its simplicity, modular structure, and extensive support for web scraping, AI integration, and database management. It enables seamless interaction between system components and rapid development.

6.2 Web Scraping

6.2.1 Selenium WebDriver

Selenium is used for dynamic browser automation to extract data from JavaScript-rendered e-commerce websites. It enables real-time navigation, scrolling, and content retrieval.

6.2.2 BeautifulSoup4

BeautifulSoup is used for parsing HTML content and extracting structured product attributes such as price, rating, and review count efficiently.

6.3 Database

6.3.1 MongoDB Atlas

MongoDB Atlas is a cloud-based NoSQL database used to store product metadata, price history, and generated reports. Its flexible schema supports semi-structured retail data.

6.3.2 PyMongo Driver

PyMongo enables secure connectivity between Python and MongoDB, supporting CRUD operations, upsert functionality, and efficient data retrieval.

6.4 AI & Agent Framework

6.4.1 Gemini 2.5 Flash

Gemini is used for quick AI analysis, generating real-time summaries and pricing insights with low latency.

6.4.2 CrewAI

CrewAI implements the multi-agent framework, allowing multiple specialized AI agents to collaborate for advanced reasoning tasks.

6.4.3 Groq Llama 3.3

Groq Llama 3.3 supports deep analytical processing and structured prompt execution for comprehensive executive-level reporting.

6.5 Dashboard & Visualization

6.5.1 Streamlit

Streamlit is used to develop the interactive web dashboard, enabling user input handling and real-time display of insights.

6.5.2 Plotly

Plotly provides interactive charts such as price comparison and trend analysis graphs.

6.5.3 Altair

Altair is used for lightweight, declarative visualization of structured retail data.

6.6 PDF Generation

6.6.1 ReportLab

ReportLab converts AI-generated insights into professionally formatted PDF reports suitable for executive presentation and documentation.

6.7 Configuration Management

6.7.1 Pydantic-Settings

Pydantic-Settings manages structured configuration loading, ensuring type-safe handling of system parameters.

6.7.2 Environment Variables (.env)

Environment variables securely store API keys and database credentials, preventing exposure of sensitive information.

7. Results & Output

The implementation of the Autonomous Retail Researcher Agent successfully demonstrates the automation of retail data collection, structured storage, AI-driven analysis, and interactive visualization. The system was tested using multiple product queries across supported e-commerce platforms to evaluate scraping accuracy, database consistency, and AI response quality. The results confirm that the platform effectively transforms raw competitor data into structured business intelligence.

7.1 Scraping and Data Collection Results

The scraper module was tested with different product categories such as electronics, fashion, and home appliances. The system successfully extracted:

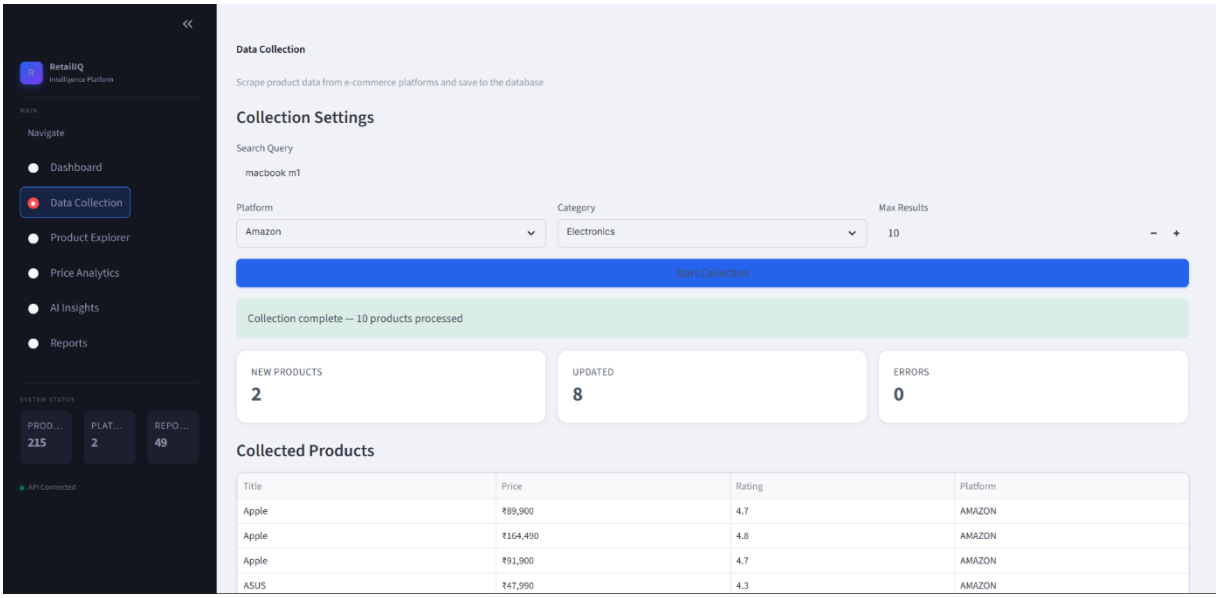
- Product title
- Current price
- Rating
- Number of reviews
- Product URL

The retry mechanism handled temporary loading failures efficiently. Duplicate entries were

avoided using SHA-256-based product identification. The extracted data was accurately normalized and stored in MongoDB without inconsistencies.

Outcome:

- Reliable dynamic scraping
- Clean and structured dataset
- Zero duplicate product entries



7.2 Database and Historical Tracking Output

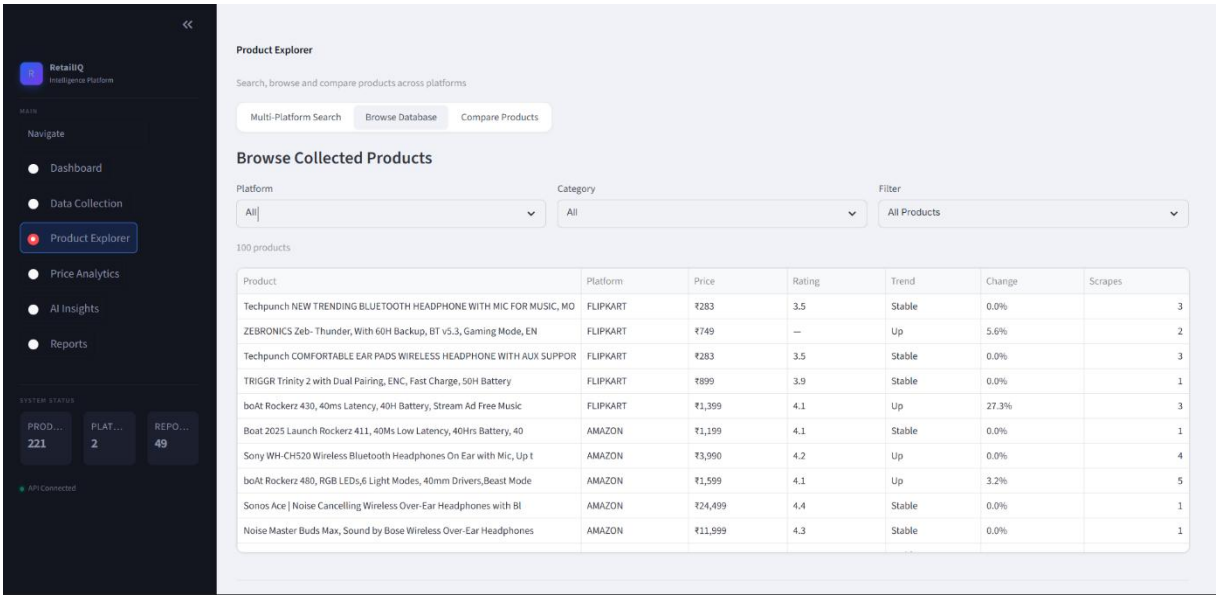
The MongoDB database stored product metadata and maintained a separate price history collection. Each new scraping cycle appended price entries with timestamps, enabling time-series tracking.

The stored data allowed:

- Historical price trend visualization
- Detection of price fluctuations
- Long-term comparative analysis

Outcome:

- Successful append-only historical storage
- Efficient query performance
- Scalable document-based data management



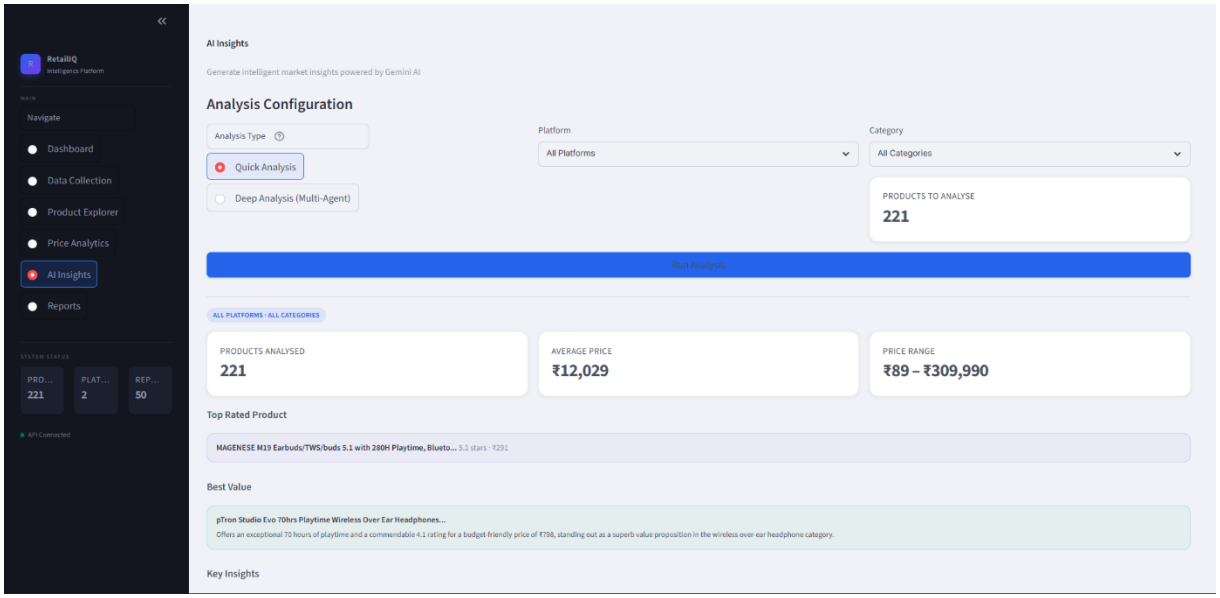
7.3 AI Analysis Output

The AI module was evaluated in two modes:

A. Quick Analysis Mode

Generated instant summaries including:

- Average price range
- Best-rated products
- Competitive positioning



B. Deep Multi-Agent Analysis Mode

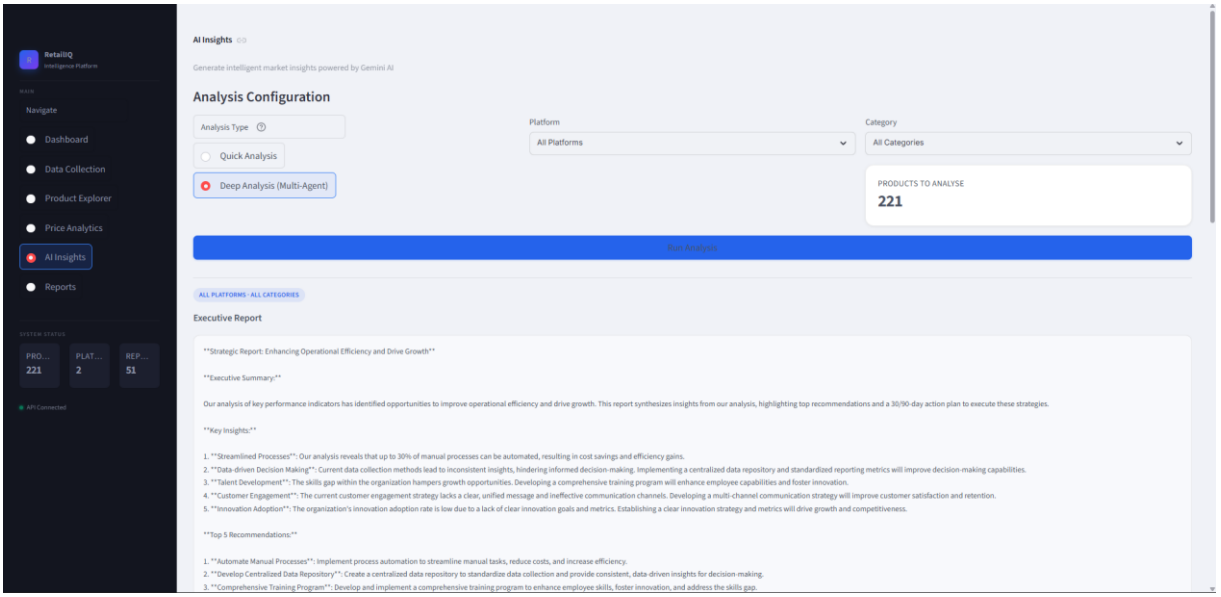
Generated structured reports containing:

- Market overview
- Risk and opportunity analysis
- Demand insights
- Strategic recommendations

The multi-agent reasoning approach produced more comprehensive insights compared to single-prompt summarization.

Outcome:

- Low-latency quick insights
- Structured executive-level analysis
- Context-aware recommendations



7.4 Dashboard and Visualization Output

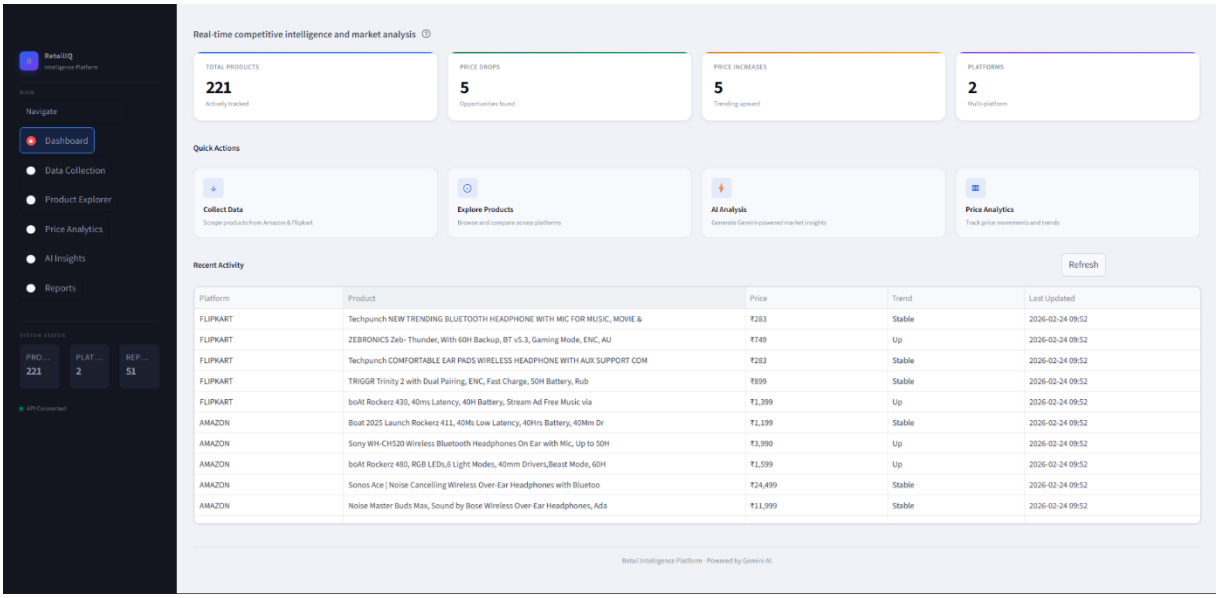
The Streamlit dashboard successfully displayed:

- KPI summary cards (average price, top-rated product, etc.)
- Interactive price comparison charts
- Historical price trend graphs
- AI-generated insight panels
- Downloadable PDF reports

Visualization libraries such as Plotly and Altair enhanced clarity and interactivity. The dashboard remained responsive due to session caching and optimized database queries.

Outcome:

- User-friendly interface
- Clear graphical interpretation of data
- Improved decision-making support



7.5 Report Generation Output

The system generated professionally formatted PDF reports containing:

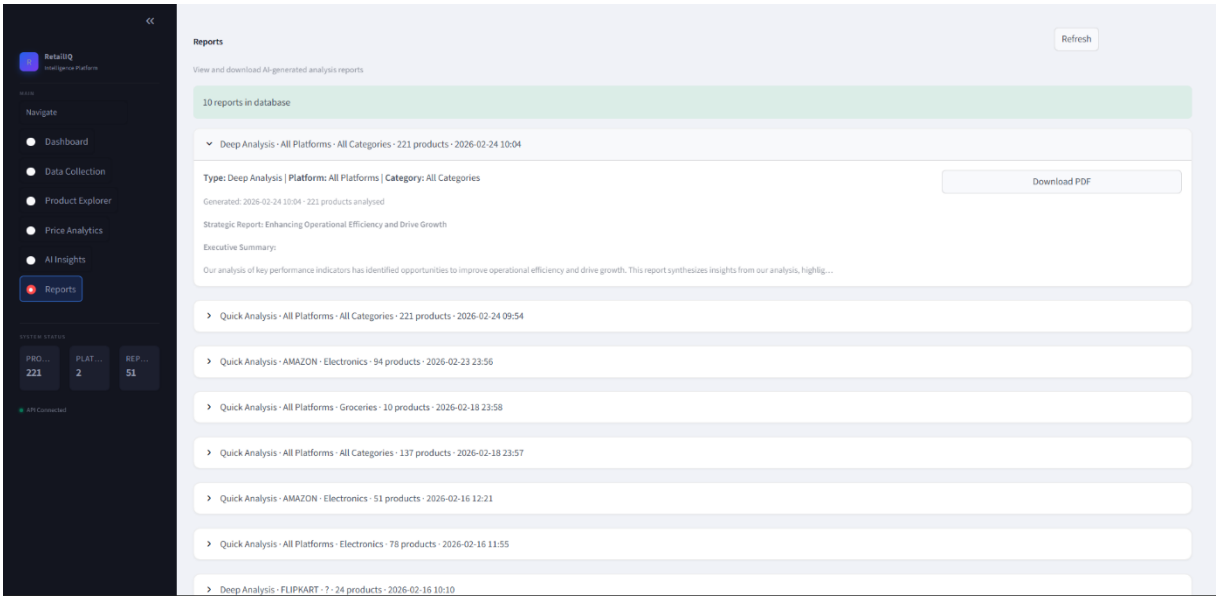
- Executive summary
- Analytical observations
- Competitive insights
- Strategic recommendations

Reports were stored in the database and could be re-downloaded without regeneration.

Outcome:

- Structured documentation
- Business-ready reporting format
- Persistent storage of analytical results

Overall, the Autonomous Retail Researcher Agent achieved its primary objective of automating retail competitor analysis. The system successfully integrates scraping, structured storage, intelligent reasoning, and visualization into a cohesive workflow. The results validate the feasibility and effectiveness of using Agentic AI frameworks for real-world business intelligence applications.



8. Conclusion

The Autonomous Retail Researcher Agent presents a comprehensive solution for automated retail intelligence by integrating web scraping, cloud-based data storage, and AI-driven analytical frameworks. The system effectively replaces traditional manual competitor analysis methods with a scalable and intelligent approach. By combining structured data acquisition with advanced reasoning models, the platform transforms raw retail information into actionable business insights.

The project successfully achieves the following key outcomes:

- **Automation of Data Collection:** Dynamic scraping using Selenium ensures real-time extraction of product details from e-commerce platforms.
- **Historical Price Tracking:** MongoDB-based append-only storage enables accurate time-series analysis of pricing trends.
- **AI-Powered Insights:** Dual-mode analysis (quick and multi-agent deep analysis) provides both rapid summaries and detailed strategic evaluation.
- **Interactive Visualization:** Streamlit dashboard with charts and KPIs enhances interpretability of complex data.
- **Professional Reporting:** Automated PDF generation supports executive-level documentation and decision-making.

The modular layered architecture enhances maintainability and scalability, ensuring that future extensions such as predictive analytics or cloud deployment can be incorporated seamlessly. Overall, the project validates the practical application of Agentic AI in retail business intelligence and demonstrates how intelligent automation can significantly improve competitive market analysis and strategic planning.

9. Future Scope & Enhancements

The Autonomous Retail Researcher Agent establishes a strong foundation for automated retail intelligence; however, several enhancements can further improve its scalability, intelligence, and enterprise readiness. The current implementation focuses on automated scraping and AI-driven analysis, but future developments can transform it into a fully production-grade retail analytics platform. The following enhancements outline potential directions for system expansion and performance optimization.

9.1 Multi-User Authentication and Role Management

Currently, the system operates as a single-user analytical platform. Future versions can incorporate a secure authentication mechanism with role-based access control. This would allow:

- Administrator and analyst role separation
- User-specific report storage
- Access control for sensitive data
- Secure login and session management

Such a feature would make the platform suitable for organizational deployment.

9.2 Automated Scheduled Scraping

Currently, scraping is user-triggered. Future versions can implement automated scheduled scraping using task schedulers (e.g., cron jobs or workflow managers). This would allow:

- Daily or hourly data collection
- Continuous price monitoring
- Historical dataset enrichment
- Automatic report generation

Such automation would transform the platform into a real-time monitoring system.

9.3 Predictive Machine Learning Models

While the current system focuses on descriptive and analytical insights, future work can incorporate predictive modeling techniques such as:

- Price trend forecasting
- Demand prediction models
- Sales performance estimation
- Market volatility analysis

Integration of machine learning models would elevate the system from reactive analytics to proactive decision support.

9.4 Real-Time Price Drop Notifications

An alert system can be implemented to notify users about significant market changes. This feature may include:

- Email or SMS alerts
- Dashboard notifications
- Threshold-based price drop triggers
- Competitor price change alerts

Real-time notification mechanisms would enhance business responsiveness.

9.5 Expansion to Additional Marketplaces

The system currently supports selected e-commerce platforms. Future enhancements may include integration with:

- International marketplaces
- Niche retail platforms
- B2B e-commerce portals

Expanding data sources would improve market coverage and analytical accuracy.

9.6 Enterprise System Integration

Integration with Enterprise Resource Planning (ERP) or Customer Relationship Management (CRM) systems would allow seamless business workflow alignment. This would enable:

- Automated pricing strategy adjustments
- Inventory optimization
- Sales strategy refinement

Such integration would bridge the gap between analytics and operational execution.

9.7 REST API Backend for Mobile and External Access

Developing a RESTful backend service would allow external applications and mobile interfaces to access retail insights. This would provide:

- Mobile dashboard access
- Third-party system integration
- API-based data sharing
- Scalability across platforms